UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

EVALUACIÓN	Obligatorio	**GRUPO**	Todos	**FECHA**	02/Set/2025
MATERIA	Diseño de Aplicaciones 2				
CARRERA					

CONDICIONES

- **Puntaje máximo:** 20 puntos
- **Puntaje mínimo:** 0 puntos
- **Fecha de entrega:** 09/Oct/2025 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf) y en GitHub.

Uso de material de apoyo y/o consulta

Inteligencia Artificial Generativa

- Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso.
- Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó.
- Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla.
- Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de "alucinaciones", los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso
- En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema

IMPORTANTE:

1. Inscribirse en la evaluación a través de gestion.ort.edu.uy.

2. Formar grupos de tres personas (no es necesario que pertenezcan al mismo dictado).

3. Subir el trabajo a Gestión (documentación) y a GitHub antes de la hora indicada (ver la sección "RECORDATORIO" al final del documento).

4. Dentro del repositorio privado en la ORG de DA2 de GitHub realizar un *merge* como *release* a *main* (GitFlow) antes de la hora indicada.

Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador de cursos o Coordinación adjunta **antes de las 20:00h** del día de la entrega, a través de los mails crosa@ort.edu.uy / posada_l@ort.edu.uy (matutino) / larrosa@ort.edu.uy (nocturno), o vía Ms Teams.

Obligatorio 1 de Diseño de Aplicaciones II

Parque Temático Virtual

Un grupo de inversores está desarrollando un **Parque Temático Virtual Interactivo** que combina atracciones físicas con experiencias digitales gamificadas. La propuesta integra un parque real, con acceso mediante pulseras NFC y códigos QR, junto a una aplicación web y móvil que permite a los visitantes registrar sus actividades, completar misiones, desbloquear logros y canjear recompensas.

El objetivo de esta plataforma es ofrecer una experiencia inmersiva y personalizada, donde cada visitante pueda interactuar tanto con las atracciones presenciales como con contenidos y retos virtuales.

Roles y funcionalidades

Administrador del parque

- **Mantenimiento de atracciones (*)**: Alta, baja y modificación de atracciones. Cada atracción tiene: nombre, tipo (montaña rusa, simulador, espectáculo, zona interactiva), edad mínima del visitante para ingreso, capacidad máxima y descripción.
- **Gestión de eventos especiales (*)**: Alta y baja de eventos temáticos (ej. “Noche de Dinosaurios”), con fecha, hora, aforo, costo adicional y lista de atracciones incluidas.
- **Mantenimiento de usuarios**: Alta de administradores, visitantes y operadores. De cada usuario se conoce: nombre, apellido, email (único) y contraseña. Para los visitantes, además se conoce su fecha de nacimiento y nivel de membresía (Estándar, Premium, VIP).
- **Estrategia de puntuación**: establecer la **estrategia activa**; el cambio impacta inmediatamente en el cálculo de puntos y en el **ranking diario**.
- **Reporte de uso de atracciones**: Cantidad de visitas por atracción entre dos fechas.
- **Sistema de ranking diario (*)**: listado de los 10 visitantes con más puntos acumulados en el día.

Visitante

- **Registro y perfil (*)**: El usuario se puede auto registrar (caso que queda como usuario “estándar”) o ser dado de alta por un administrador. El usuario puede modificar su propio perfil, excepto por su nivel de membresía.
- **Compra de tickets (*)**: seleccionar fecha de visita y tipo de entrada (general o evento especial), lo que genera un código QR único (para simplificación tomamos el QR como un GUID). En caso de los eventos especiales, se debe tener en cuenta el aforo para limitar la venta de entradas.
- **Registro automático de atracciones visitadas**: ingreso mediante pulsera NFC (para simplificación tomamos que el NFC es un GUID que identifica al usuario, diferente al GUID del QR) o el escaneo del QR de su ticket, en la entrada de una atracción.

Operador de atracción

- **Control de acceso a atracciones**: escaneo de tickets (QR) o pulseras (NFC) para validar si el visitante tiene un ticket válido para la atracción. La validez depende del tipo de ticket, fecha, hora, atracciones que incluye (en caso de evento especial), aforo, y edad mínima.
- **Registro de egreso**: marcar la salida del visitante de la atracción.
- **Visualización de aforo**: mostrar aforo actual y capacidad restante. En caso de que el aforo esté completo, no se permite el ingreso al visitante a la atracción.
- **Gestión de incidencias:** alta y baja de incidencias de una atracción (ej. fuera de servicio). Mientras exista una incidencia activa, la atracción no permite ingresos de visitantes.

Gamificación (puntuación y ranking)

- Definir una **familia de estrategias de puntuación para calcular los puntos del visitante** al registrar accesos y completas misiones. Ejemplos:
  - PuntuacionPorAtraccion: otorga puntos fijos por tipo de atracción.
  - PuntuacionCombo: bonifica cadenas de atracciones diferentes en ≤ N minutos.
  - PuntuacionPorEvento: multiplica puntos durante eventos especiales.
- Requerimientos:
  - Debe existir una **estrategia activa** seleccionable en tiempo de ejecución.
  - Endpoints para **listar** y **cambiar** la estrategia activa.
  - El **ranking diario** se calcula usando la estrategia activa del día.
  - La arquitectura debe permitir **agregar nuevas estrategias sin modificar código cliente.**

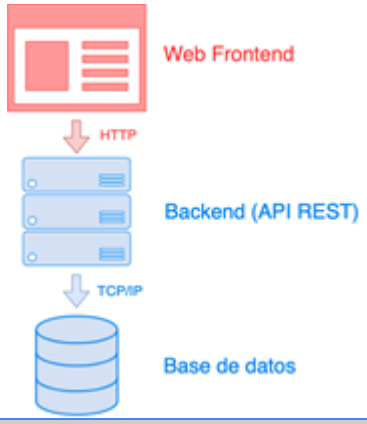
Registro de fecha y hora

- Para facilitar las pruebas del sistema, se debe poder **configurar la fecha y hora actual**, mediante un endpoint público que reciba el valor en formato "YYYY-MM-DDTHH:MM", por ejemplo "2025-09-02T14:45".
- Esta fecha/hora configurada se usará en toda la aplicación.
- Si nunca se configuró, se toma por defecto, la fecha/hora del sistema operativo.
- Debe existir también un endpoint público para **consultar** la fecha y hora actual en uso.

Autenticación y endpoints públicos

- El sistema debe contar con login para usuarios (Administrador, Operador y Visitante).
- Se deberán implementar endpoints públicos para: **auto-registro de visitante**, **login** y **gestión/consulta de fecha y hora**.
- Todos los demás endpoints serán privados y requerirán autenticación con token enviado en el header de la petición.

Para resolver el problema se definió la siguiente **arquitectura de alto nivel:**



Artefacto	Descripción
Base de datos	Base de datos relacional (SQL Server) en donde se almacenan los datos de la aplicación.
Api REST + Backend	Toda interacción con el repositorio de datos se realiza mediante un API REST, la que ofrece operaciones para resolver todo lo necesario y el back end donde se implementa la lógica de negocio.
Web Frontend	Aplicación que permite a los usuarios registrarse e interactuar con la aplicación.

Alcance de la primera entrega

Considerando la arquitectura mencionada en la sección anterior, para esta primera **entrega se debe implementar una API REST** que ofrezca todas las operaciones que el alumno considere necesarias para soportar la aplicación descrita en este documento.

Dicho de otra manera, se debe implementar **todo el backend** de la aplicación (lógica de negocio), pero **no el frontend** de la misma (interfaz de usuario). Se espera que, mediante la descripción de la aplicación, y complementando con consultas a aulas cuando considere necesario, el alumno sea capaz de identificar las operaciones que el API debe ofrecer.

Dado que la aplicación no tendrá interfaz de usuario, la forma de interactuar con la misma será mediante un cliente HTTP. Se espera que se utilice **Postman** (https://www.getpostman.com/) como cliente, ya que se debe entregar una exportación de una colección de Postman con los end-points definidos para poder probar. Debe tener todas las llamadas a la API preparadas y utilizar la parametrización de Postman en cuanto a la URL a utilizar, token en el header, etc. para no tener que escribir todas las llamadas nuevamente en la demostración.
En caso de no entregar dicha colección, los docentes pueden optar por la no corrección de la funcionalidad. Ver la rúbrica de evaluación en las secciones más abajo.

Implementación

Se debe entregar una implementación del API REST necesaria para soportar la aplicación que se describe. La entrega debe contener una solución de Visual Studio que agrupe todos los proyectos implementados.
La solución debe incluir el código de las pruebas automáticas. Se requiere escribir los casos de prueba automatizados con MSTests, documentando y justificando las pruebas realizadas.
Se espera que la aplicación se entregue con una base de datos con datos de prueba, de manera de poder comenzar las pruebas sin tener que definir una cantidad de datos iniciales. Dichos datos de prueba deben estar adecuadamente especificados en la documentación entregada.

Tecnologías y herramientas de desarrollo

- Microsoft VSCode/ Visual Studio
- Microsoft SQL Server Express 2017
- Postman
- .NET SDK 8.0 / ASP.NET 8.0(C#) / Entity Framework Core 8.0
- Astah o cualquier otra herramienta UML 2

Instalación

El costo de instalación de la aplicación debe de ser mínimo y documentado adecuadamente. **Agregar esta guía de instalación en la carpeta "Datos" del repositorio.**

NOTA: La totalidad y detalle de los requisitos serán relevados a partir de consultas en el foro correspondiente en aulas. Para evitar complejidades innecesarias se realizaron simplificaciones al dominio del problema real.

Independencia de librerías

Se debe diseñar la solución que al modificar el código fuente minimice el impacto del cambio en los componentes físicos de la solución. Debe documentar explícitamente como su solución cumple con este punto. Cada paquete lógico debe ser implementado en un *assembly* independiente, documentando cuáles de los elementos internos al paquete son públicos y cuáles privados, o sea cuáles son las interfaces de cada *assembly*.

Persistencia de los datos

La empresa requiere que todos los datos del sistema sean persistidos en una base de datos. Así, cuando se ejecute la aplicación se empezará con los datos cargados con el último estado guardado antes de cerrar la aplicación.

El diseño debe contemplar el modelado de una solución de persistencia adecuada para el problema utilizando Entity Framework (*Code First*).

Se espera que como parte de la entrega se incluya dos respaldos de la base de datos: uno vacío y otro con datos de prueba. Se debe entregar el script *.sql* para ambas bases de datos.

Mantenibilidad

La propia empresa eventualmente hará cambios sobre el sistema, por lo que se requiere un alto grado de mantenibilidad, flexibilidad, calidad, claridad del código y documentación adecuada.

Por lo que el desarrollo de todo el obligatorio debe cumplir:

- Estar en un repositorio **Git**.
- Haber sido escrito utilizando **TDD** (desarrollo guiado por pruebas) lo que involucra otras dos prácticas: escribir las pruebas primero (Test First Development) y Refactoring. De esta forma se utilizan las pruebas unitarias para dirigir el diseño.
  Es necesario utilizar **TDD únicamente** para el *back end* y la *API REST*, no para el desarrollo del *front end*.
  Se debe utilizar un framework de Mocking (como Moq, https://www.nuget.org/packages/moq/) para poder realizar pruebas unitarias sobre la lógica de negocio. En caso de necesitar hacer pruebas para el acceso a datos, deberán de hacerlo con el paquete EF Core Sqlite (https://www.nuget.org/packages/Microsoft.EntityFrameworkCore.Sqlite).
- Cumplir los lineamientos de **Clean Code** (capítulos 1 al 10, y el 12), utilizando las técnicas y metodologías ágiles presentadas para crear código limpio.

Control de versiones

La gestión del código del obligatorio debe realizarse utilizando UN ÚNICO repositorio Git de **Github**, apoyándose en el flujo de trabajo recomendado por **GitFlow** (https://nvie.com/posts/a-successful-git-branching-model/).

Dicho repositorio debe **pertenecer a la organización de GitHub "IngSoft-DA2"**

(https://github.com/IngSoft-DA2), en la cual deben estar todos los miembros del equipo.

Al realizar la entrega se debe realizar un *release* en el repositorio y la rama *main* no debe ser afectada luego del hito que corresponde a la entrega del obligatorio.

Evaluación (20 puntos)

Las condiciones de entrega serán evaluadas como si se le estuviese entregando a un cliente real: prolijidad, claridad, profesionalismo, orden, etc.

La entrega debe realizarse antes de las 21:00 horas del día de la entrega, tanto en **gestion.ort.edu.uy** como en el repositorio de GitHub del equipo. A continuación, se detallan los requisitos para cada uno:

1. **Entrega en Gestión (gestion.ort.edu.uy):**
   - La documentación completa debe estar en formato PDF e incluir el modelado UML. Es fundamental que todos los diagramas sean claros y legibles.

La documentación para entregar debe dividirse en **4 documentos**:

- Descripción del diseño (PDF que no debe superar las 20 páginas).
- Evidencia del diseño y especificación de la API.
- Evidencia de la aplicación de TDD y Clean Code (PDF que no debe superar las 15 páginas).
- Evidencia de la ejecución de las pruebas de la API con Postman (PDF que no debe superar las 15 páginas).

Todos los documentos deben cumplir con los siguientes elementos del Documento 302 de la facultad (http://www.ort.edu.uy/fi/pdf/documento302facultaddeingenieria.pdf):

- Capítulo 3, secciones 3.1 (sin la leyenda), 3.2, 3.5, 3.7, 3.8 y 3.9.
- Capítulo 4 (salvo 4.1).
- Capítulo 5.
- Se **debe incluir en las portadas de cada documento la URL al repositorio del equipo**

2. **Entrega en el Repositorio GitHub del equipo:**
   - **Carpeta raíz**: Contendrá el código fuente de la aplicación, junto con todo lo necesario para compilar y ejecutar la aplicación.
   - **Carpeta "Aplicación"**: Incluirá la aplicación compilada en modo release y todos los elementos necesarios para realizar la instalación (deploy) de la misma.
   - **Carpeta "Datos"**: Incluirá el archivo .sql. Se debe entregar una versión de la base de datos vacía y otra con datos de prueba.

Rúbrica de evaluación

Evaluación de	

Descripción del diseño.	El objetivo del documento es demostrar que el equipo pudo diseñar y documentar el diseño de la solución. La documentación debe pensarse para que un tercero (corrector) pueda comprender la estructura y los principales mecanismos del código. O sea, debe servir como guía para entender el código y los aspectos más relevantes del diseño y la implementación.
	
	El documento **debe** organizarse siguiendo el modelo 4+1 haciendo énfasis en las vistas de diseño e implementación. Este documento no debe incluir paquetes o componentes de los proyectos de prueba.
	
	Elementos para evaluar:
	• Descripción general del trabajo (qué hace la solución) y errores conocidos (*bugs* o funcionalidades no implementadas).
	• Diagrama general de paquetes (namespaces) mostrando los paquetes organizados por capas (*layers*) y sus dependencias. En caso de que haya paquetes anidados, se debe utilizar el conector de *nesting*, mostrando la jerarquía de dichos paquetes.
	• Cada paquete debe tener una breve descripción de responsabilidades y un diagrama de clases.
	• Descripción de jerarquías de herencia utilizadas (en caso de que así haya sido)
	• Modelo de tablas de la estructura de la base de datos.
	• Para todas las funcionalidades cuyo modelado y explicación contribuyan al objetivo de este documento —demostrar que el equipo pudo diseñar y documentar la solución de forma que un tercero (corrector) pueda comprender la estructura y los principales mecanismos del código— se deberán incluir:
	  ○ Diagramas de interacción que muestran las clases involucradas en estas funcionalidades y las interacciones para lograr la funcionalidad.
	  ○ Estos diagramas deben mostrar de manera clara todas las clases y objetos involucrados y el orden de los mensajes o llamadas entre ellos.
	  ○ Cada diagrama debe representar con precisión cómo las instancias colaboran para lograr la funcionalidad, incluyendo creación de objetos, invocaciones de métodos y retornos relevantes.
	  ○ El diagrama debe estar alineado con el código y permitir que un tercero pueda mapear fácilmente las interacciones representadas con la implementación real.
		6

- Justificación del diseño explicando mediante texto y diagramas, haciendo énfasis en:
  - La utilización de mecanismos de inyección de dependencias, fábricas, patrones y principios de diseño, etc. Discutir brevemente cómo estos mecanismos apoyan la mantenibilidad de la aplicación.
  - Sus propias decisiones propias de diseño que hacen a su obligatorio. Explicación de estructuras, mecanismos o algoritmos que tienen impacto en la mantenibilidad o desempeño de la solución.
  - Descripción del mecanismo de acceso a datos utilizado.
  - Descripción del manejo de excepciones.
- Diagrama de implementación (componentes) mostrando las dependencias entre los mismos. Justificar el motivo por el cual se dividió la solución en dichos componentes.

Se considera como aceptable si:

- Los diagramas presentan el uso adecuado de la notación UML.
- La estructura de la solución representa la descomposición lógica (módulos) y de proyectos de la aplicación.
- Las vistas de módulos, de componentes, modelo de datos y los comportamientos documentados sirven como guía para la comprensión del código implementado.
- La taxonomía de paquetes y clases en los diagramas respeta las convenciones de nombre de C# utilizada en la implementación.
- Se justifica y explica el diseño en base al uso de principios y patrones de diseño. Los mismos se implementan correctamente a partir de su objetivo y teniendo en cuenta sus ventajas y desventajas para favorecer o inhibir la calidad de la solución. El objetivo es describir el impacto de las decisiones tomadas para mejorar la mantenibilidad del sistema. No debe limitarse una descripción de lo realizado.
- Correcto manejo de las excepciones e implementación de acceso a base de datos.
- Se describen claramente los errores conocidos.
- La documentación se encuentra bien organizada, es fácil de leer y su formato corresponde con los ítems indicados del documento 302.
- La documentación no debe superar las 20 páginas.

Evidencia del diseño y especificación de la API.	Documento describiendo la API conteniendo:
	• Discusión de los criterios seguidos para asegurar que la API cumple con los criterios REST.
	• Descripción del mecanismo de autenticación de requests.
	• Descripción general de códigos de error (1xx, 2xx, 4xx, 3xx, 5xx).
	• Descripción de los *resources* de la API.
	  ○ URL base.
	  ○ Para cada *resource* describir:
	  ▪ Resource.
	  ▪ Description.
	  ▪ Endpoints (Verbo + URI).
	  ▪ Parameters.
	  ▪ Responses (para todos los códigos de estado).
	  ▪ Headers.
	
	Se considera como aceptable si:
	• El diseño de la API REST se basa en buenas prácticas de la industria. Por ejemplo:
	  ○ https://developer.spotify.com/documentation/web-api/reference/
	  ○ https://docs.microsoft.com/en-us/azure/architecture/best-practices/api-design
	  ○ Web API Design: The Missing Link by apigee.
	• Para generar la documentación se puede utilizar herramientas como swaggerHub (https://swagger.io/tools/swaggerhub/) que permiten generar la especificación de la API en formato electrónico.
	• La documentación se encuentra bien organizada, es fácil de leer y su formato corresponde con los ítems indicados del documento 302.
		4.5

Evidencia de Clean Code y de la aplicación de TDD	El trabajo se debe desarrollar en su totalidad siguiendo las prácticas de Clean Code y el enfoque de desarrollo TDD. Queda explícitamente prohibido incorporar fragmentos de código generados automáticamente por herramientas de IA sin revisión, adaptación y justificación técnica propia. La autoría se evaluará tanto en la estructura del diseño como en las decisiones de implementación. El desarrollo del trabajo debe realizarse utilizando exclusivamente los conceptos, metodologías y tecnologías presentados en clase, aplicados de forma coherente e integrada en la solución. Con el fin de demostrar la correcta ejecución de TDD, para las funcionalidades marcadas con (*), es necesario **evidenciar** la aplicación de esta técnica mostrando la evolución de las pruebas y del código a través de commits documentados. El código debe ajustarse a las prácticas recomendadas por Clean Code. Estas apuntan a que el código sea legible por un tercero. La mejor evidencia de la aplicación de Clean Code es que un tercero (los docentes), pueda leer el código y entender la intención detrás de cada clase, método y prueba con el menor esfuerzo posible. Se valorará la presencia de abstracciones bien definidas, cohesión interna en las clases, bajo acoplamiento entre módulos y encapsulamiento correcto, como indicadores de un diseño de calidad más allá de la mera sintaxis. **Resultado de la ejecución de las pruebas** Se debe entregar evidencia del código de pruebas automáticas (unitarias y de integración), el reporte de la herramienta de cobertura y un **análisis interpretativo** de los resultados. Como parte de la evaluación, se revisará el nivel de cobertura de los tests sobre el código entregado, por lo que se debe incluir el reporte y el análisis de cualquier desviación respecto a los valores esperados. Ítems para evaluar para cada una de las funcionalidades indicadas: • Descripción de la estrategia de TDD seguida (inside – out o outside - in). • Informe de cobertura para todas las pruebas desarrolladas. • Es importante mantener una clara separación de los proyectos de prueba y los de la solución. • **Para las funcionalidades especificadas como prioritarias (*) se debe tener en cuenta:**	4.5

- Dejar evidencia, mediante los commits, que se aplicó TDD mostrando la evolución del ciclo de TDD.
- Análisis de las métricas de cobertura. Se espera que la métrica para las pruebas del código indicado la cobertura se encuentre en un valor entre 90% y 100% de líneas de código. Se debe realizar un análisis de cualquier desvío de estos valores.

Se considera como aceptable si:

- El código debe ajustarse a las buenas prácticas de Clean Code. https://www.planetgeek.ch/wp-content/uploads/2013/06/Clean-Code-V2.1.pdf
- Un tercero conocedor de Clean Code pueda leer el código sin dificultad.
- El informe de las pruebas sirve como evidencia de la correcta aplicación de desarrollo guiado por las pruebas (TDD) y técnicas de refactoreo de código. Se justifica claramente el motivo por los cuales se obtuvieron los valores registrados.
- La documentación se encuentra bien organizada, es fácil de leer y su formato corresponde con los ítems indicados del documento 302.
- La documentación no debe superar las 15 páginas.
- Se perderán todos los puntos de esta sección si no se hace uso de Pull Request y de las Github Actions pactadas. Más información de configuración y uso:
  - Repositorio y Github Actions https://github.com/IngSoft-DA2/DA2-Tecnologia/tree/repo-configuration
  - Las Github Actions tienen que ser ejecutadas usando self hosted runners los cuales deben ser configuradas por cada integrante del equipo https://github.com/IngSoft-DA2/DA2-Tecnologia/blob/main/github-self-hosted-runner-docker.md
  - Pull Request: https://github.com/IngSoft-DA2/DA2-Tecnologia/tree/pr-creation

Evidencia de la ejecución de las pruebas de la API con Postman.	Se debe generar las pruebas en una colección de Postman **para <u>todas las funcionalidades</u>**, dicho archivo se debe incluir en el repositorio en la carpeta "Datos". Para las funcionalidades marcadas con (*) **se debe realizar casos de prueba** elaborados, incluyendo: valores inválidos, valores límites, ingreso de tipos de datos erróneos, datos vacíos, datos nulos, omisión de campos obligatorios, campos redundantes, Body vacío {}, formato de los mensajes inválidos, pruebas de las reglas del negocio como alta de elementos existentes, baja de elementos inexistentes, etc. Se debe entregar un video que muestre evidencia del resultado de ejecutar los casos de prueba anteriores especificados para la API con Postman para **todas las funcionalidades**. La evidencia **<u>debe</u>** registrarse realizando **un único video publicado en YouTube**, con una **duración máxima de 15 minutos** y en el cual **se pueda apreciar claramente la ejecución de todas las pruebas**. El link a este video debe incluirse en este documento y se debe verificar que se hayan compartido correctamente y que un tercero pueda verlos. Se considera como aceptable si: • El documento deja claro que la prueba que se ejecutó y cuál fue el resultado obtenido. • La documentación se encuentra bien organizada, es fácil de leer y su formato corresponde con los ítems indicados del documento 302. • La documentación no debe superar las 15 páginas.	5

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

- **Obligatorios**

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem "Evaluaciones" y la instancia de evaluación correspondiente, que figura bajo el título "Inscripto".
3. Para iniciar la entrega hacé clic en el ícono:
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en "Agregar". El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en "Crear equipo".

   Cualquier integrante podrá:

   • Modificar la integración del equipo.

   • Subir el archivo de la entrega.
5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en "Subir" para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

   Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.
6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador de cursos o Coordinación adjunta antes de las 20:00h del día de la entrega, a través de los mails crosa@ort.edu.uy / posada_l@ort.edu.uy (matutino) / larrosa@ort.edu.uy (nocturno), o vía Ms Teams.